



# Light-SQL: A Small Language Model Based Approach Using In-Context Learning for Natural Language to SQL

Kieu-Trang Nguyen<sup>1</sup> and Quang-Hung Le<sup>2</sup>(✉)

<sup>1</sup> Department of Mathematics and Statistics, Quy Nhon University, Gia Lai, Vietnam  
trang4554110021@st.qnu.edu.vn

<sup>2</sup> Department of Information Technology, Quy Nhon University, Gia Lai, Vietnam  
lequanghung@qnu.edu.vn

**Abstract.** The transformation of natural language queries into SQL commands (NL-to-SQL) is a critical task for democratizing data access. While large language models (LLMs) have shown promising results, their deployment is often hindered by high computational costs and resource demands. This paper explores the potential of high-performance small language models (SLMs) as efficient alternatives. We propose a novel approach tailored for SLMs to address the NL-to-SQL task. Particularly, our approach leverages in-context learning (ICL) augmented by semantic retrieval to dynamically select relevant few-shot examples without the need for fine-tuning. Experimental results on the complex CORDIS scientific dataset demonstrate that the proposed method achieves a competitive execution accuracy of 41% with 3-shot learning. This performance surpasses several baselines, proving that SLMs combined with prompt engineering can effectively handle complex database schema logic on a single consumer-grade GPU. We share our experimental setup of this work on Github repository (<https://github.com/gilchea/Light-SQL>).

**Keywords:** Natural Language to SQL · Small Language Models · Prompt Engineering · In-Context Learning · Semantic Retrieval

## 1 Introduction

In the era of big data, the ability to interact with and extract information from relational databases is an essential skill. However, effective data retrieval typically requires proficiency in structured query language (SQL), creating a significant barrier for the majority of non-technical users. The NL-to-SQL task aims to bridge this gap by converting user questions in natural language into executable SQL queries [1–3], thereby democratizing access to structured data.

While the advent of LLMs [4] has revolutionized natural language processing, offering impressive capabilities in code generation and semantic understanding [5], deploying these massive models presents substantial challenges. They require enormous computational resources, resulting in high operational costs

and latency issues that make them impractical for many real-time or edge-computing applications. Furthermore, exposing private enterprise schemas to external LLM APIs raises significant data privacy concerns.

Consequently, recent research has shifted towards SLMs that maintain competitive performance while being significantly lighter [6–8]. These models have emerged as powerful alternatives, offering reasoning capabilities comparable to larger models but deployable on constrained hardware. In this paper, we investigate whether such SLMs can effectively handle the complex logic required for the NL-to-SQL task. To maximize the potential of SLMs without expensive fine-tuning, prompt engineering strategies are crucial [9]. In particular, ICL allows the model to learn a new task from a few examples provided directly in the prompt [10, 11]. Since the effectiveness of ICL heavily depends on the quality of these examples, we employ advanced semantic retrieval mechanisms (often referred to as retrieval-augmented generation (RAG) [12]) to dynamically fetch the most relevant demonstrations for a specific query, thereby enhancing the model’s reasoning context.

To the best of our knowledge, this is the first work to explore the use of SLMs for NL-to-SQL via ICL augmented by dynamic semantic retrieval, eliminating the need for fine-tuning. The main contributions of our work are summarized as follows:

1. We propose Light-SQL, a lightweight yet effective framework tailored for SLMs to address the NL-to-SQL task.
2. We integrate a semantic retrieval module to dynamically select ICL examples based on vector similarity, replacing static example selection strategies.
3. We conduct an empirical analysis of model performance across different ICL scenarios to evaluate execution accuracy.
4. We demonstrate that our proposed SLM-based approach achieves 41% execution accuracy on the CORDIS dataset [2], outperforming several baselines such as T5-Base [13] and SmBoP [14].

The rest of the paper is structured as follows. Section 2 reviews related work, while Sect. 3 defines the problem and its challenges. Section 4 details our methodology, including the proposed framework and retrieval strategies. Section 5 presents the experimental results, and finally, Sect. 6 concludes and outlines future work.

## 2 Related Work

The evolution of NL-to-SQL systems has progressed from early rule-based approaches to sophisticated neural architectures [15]. The development of neural semantic parsers was significantly accelerated by large-scale benchmarks such as Spider [1] and ScienceBenchmark [2]. Notable models like SmBoP [14] and RAT-SQL [16] utilized graph neural networks to align natural language questions with database schemas. Meanwhile, sequence-to-sequence transformers like T5

[13] and PICARD [17] treated the task as a translation problem, achieving significant improvements in accuracy through constrained decoding.

With the advent of LLMs [4], the paradigm shifted from supervised fine-tuning to prompting techniques. Foundation models such as GPT-4 [5] demonstrated the ability to generate complex SQL queries via zero-shot or few-shot inference. Specialized frameworks like DIN-SQL [18] and DAIL-SQL [19] further enhanced performance by decomposing the generation process and optimizing example selection. However, the immense computational cost and privacy concerns associated with these massive models limit their deployment in resource-constrained environments.

To address the aforementioned challenges, recent research has focused on SLMs as efficient alternatives. Models such as Mistral 7B [6], Llama 2 [20], and Phi-3 [8] are designed to retain strong reasoning capabilities within a compact parameter space. Techniques like QLoRA [21] enable these models to be fine-tuned or deployed on consumer-grade hardware with minimal performance degradation. Despite their potential, SLMs often struggle with the complex schema linking required for domain-specific NL-to-SQL tasks compared to their larger counterparts, as highlighted in recent benchmarks [3].

Prompt engineering [9] and ICL [10] have emerged as critical strategies to enhance the reasoning capabilities of frozen language models. Methods such as chain-of-thought prompting [11] and self-consistency [22] encourage models to generate intermediate reasoning steps. In the context of NL-to-SQL, RAG [12] is frequently employed to dynamically select relevant few-shot examples using dense retrievers [23] or sentence embeddings [24,25]. This paper builds upon these advancements by integrating semantic retrieval with efficient SLMs to bridge the gap between computational efficiency and accuracy in complex query generation.

### 3 Problem Definition

The problem of NL-to-SQL is formally defined as translating a user’s natural language question into a syntactically correct and semantically consistent SQL query. Let  $q$  denote a natural language question and  $\mathcal{S} = \{\mathcal{T}, \mathcal{C}, \mathcal{R}\}$  represent a relational database schema, where  $\mathcal{T}$ ,  $\mathcal{C}$ , and  $\mathcal{R}$  denote the sets of tables, columns, and foreign key relations, respectively. The objective is to approximate a mapping function  $f$  such that:

$$f : (q, \mathcal{S}) \rightarrow s^* \quad (1)$$

where  $s^*$  represents the predicted SQL query. The correctness of  $s^*$  is evaluated by its execution on a database instance  $\mathcal{D}_{\mathcal{S}}$ . Ideally, the execution result  $s^*(\mathcal{D}_{\mathcal{S}})$  should match the ground truth answer  $A$  derived from  $q$ :

$$q, \mathcal{S} \xrightarrow{f} s^*, \quad \text{where } s^*(\mathcal{D}_{\mathcal{S}}) = \text{Answer}(q) \quad (2)$$

**Challenges in NL-to-SQL.** Unlike simple sequence-to-sequence translation tasks, NL-to-SQL presents unique challenges that hinder the performance of

both traditional parsers and modern LLMs. First, *schema linking* requires the model to accurately map natural language tokens to specific schema elements (tables and columns), a task often complicated by lexical mismatches between user terminology and database identifiers [3, 26]. Second, the *structural complexity* of real-world databases necessitates intricate SQL operations involving multi-table JOINS, nested sub-queries, and aggregations, which are difficult to infer from ambiguous natural language descriptions [1, 2]. Finally, models must possess strong *generalization* capabilities to adapt to unseen database schemas in cross-domain settings without specific training on the target domain [19]. To illustrate the complexity of the task, we consider the schema provided in Listing 1.1 involving project funding. To answer the question regarding “total cost”, the model must correctly infer the join path between the **projects** and **people** tables based on the defined foreign key relationship.

**Listing 1.1.** Illustrative Database Schema and NL-to-SQL Mapping

```

### Database Schema S
CREATE TABLE projects (
  unics_id NUMBER PRIMARY KEY,
  title TEXT,
  total_cost NUMBER,
  principal_investigator NUMBER,
  FOREIGN KEY (principal_investigator) REFERENCES people(unics_id)
);
CREATE TABLE people (
  unics_id NUMBER PRIMARY KEY,
  full_name TEXT
);

### Input Question q
Show the names of people having projects with a total cost not equal to
4538143

### Target SQL s*
SELECT people.full_name
FROM people
JOIN projects ON people.unics_id = projects.principal_investigator
WHERE projects.total_cost != 4538143.0;

```

## 4 Methodology

Our proposed framework for NL-to-SQL translation leverages the computational efficiency of SLMs while enhancing their reasoning capabilities through advanced prompt engineering and ICL.

### 4.1 Overview of the Light-SQL Framework

We propose a lightweight yet effective framework named Light-SQL, tailored for SLMs to address the NL-to-SQL task. Unlike large-scale models that rely on massive parameter counts, our approach optimizes performance through a retrieval-augmented generation pipeline. As illustrated in Fig. 1, the framework consists of three core components:

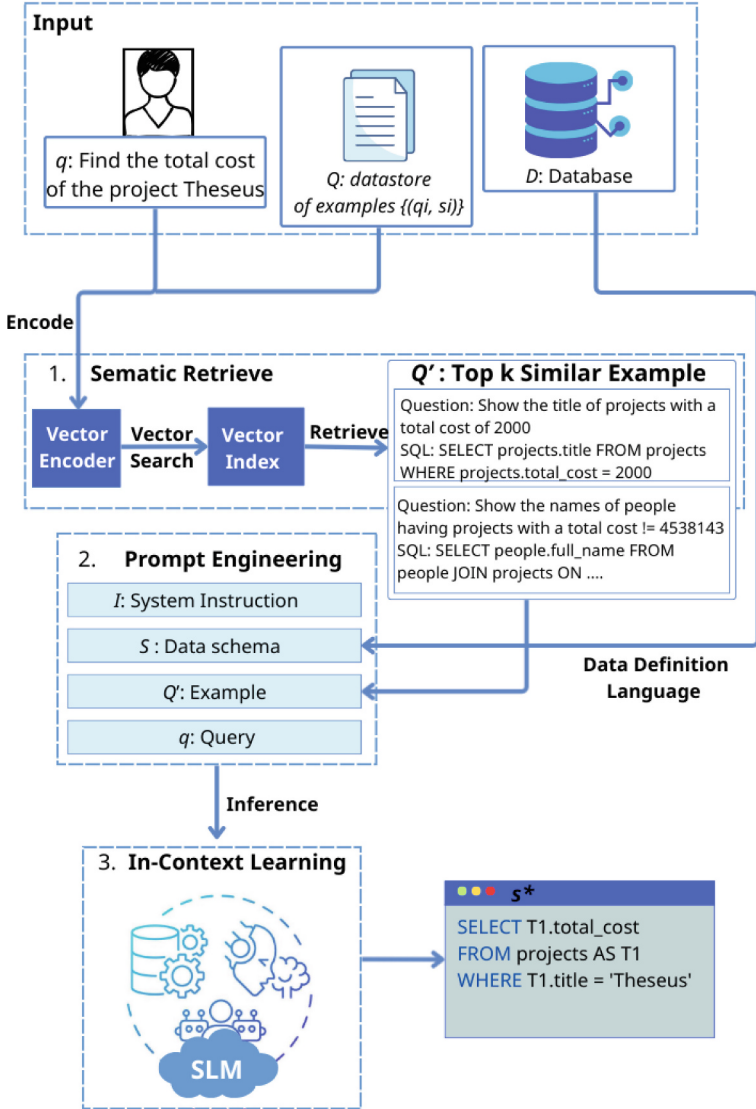


Fig. 1. Overview of proposed method.

1. *Semantic Retrieval* module that identifies relevant historical query pairs from a vector database.
2. *Prompt Engineering* module that constructs a context-aware input containing schema definitions and retrieved demonstrations.
3. *In-Context Learning* mechanism that guides the SLM to generate the target SQL query without parameter updates. This design aims to balance com-

putational efficiency with the complex reasoning required for schema linking and SQL syntax generation [9, 19].

## 4.2 Semantic Retrieval

To enhance the model’s ability to generalize across different query types, we employ a retrieval-based strategy commonly referred to as RAG [12]. Given an input natural language question  $q$ , the system queries a pre-indexed data store of examples  $\mathcal{Q} = \{(q_i, s_i)\}$ . We utilize a dense encoder, such as Sentence-BERT [24], to map both the target question and candidate questions into a shared high-dimensional vector space. While the fundamental retrieval mechanism relies on measuring the proximity between the target question embedding  $v_q$  and candidate embeddings  $v_{q_i}$  (e.g., via cosine similarity), the specific criterion for selecting the most effective demonstrations  $\mathcal{Q}'$  is a critical hyperparameter. For instance, in a similarity-based setting, if the user asks “*Find the total cost of the project Theseus*”, the retriever identifies semantically similar structural queries, such as questions about “total funding” or “project budgets” from the training set, even if the specific entities differ. The top- $k$  most similar pairs are then retrieved to serve as few-shot demonstrations, effectively grounding the model’s generation in valid SQL patterns [23]. We systematically present different selection strategies utilized within this retrieval framework in Sect. 4.4.

## 4.3 Prompt Engineering

Prompt engineering serves as the critical interface between the structured database schema and the generative capabilities of the SLM. In our framework, the prompt structure is designed to efficiently utilize the limited context window of SLMs while minimizing hallucination and syntax errors. We adopt a strict template where the complete prompt  $\sigma$  is constructed as follows:

$$\sigma(q, \mathcal{S}, \mathcal{I}, \mathcal{Q}') = q \oplus \mathcal{S} \oplus \mathcal{I} \oplus \mathcal{Q}' \quad (3)$$

The function  $\sigma(q, \mathcal{S}, \mathcal{I}, \mathcal{Q}')$  synthesizes the target question  $q$  with three key context components. First, the system instruction ( $\mathcal{I}$ ) explicitly establishes the model’s persona as an SQL expert and imposes strict output constraints to prevent non-SQL explanations. Second, the schema context ( $\mathcal{S}$ ) is serialized using data definition language rather than simple table listings; this format preserves critical metadata such as column types and foreign key relationships, enabling the model to infer correct join paths. Finally, these are augmented with a set of  $k$  few-shot examples  $\mathcal{Q}'$  retrieved via the strategy detailed in Sect. 4.4. Listing 1.2 illustrates the concrete implementation of this prompt structure.

**Listing 1.2.** Structure of the Instruction-Tuned Prompt

```
### Instruction:
You are an expert in SQL. Generate a valid SQL query for the following
question based on the provided schema.
```

```

### Schema:
CREATE TABLE projects (
  unics_id NUMBER PRIMARY KEY,
  title TEXT,
  total_cost NUMBER,
  principal_investigator NUMBER,
  FOREIGN KEY (principal_investigator) REFERENCES people(unics_id)
);
... [Other Tables] ...

### Few-shot Examples:
... [Retrieved Examples] ...

### Question:
Find the total cost of the project Theseus

### Response:
% SELECT projects.total_cost FROM projects WHERE projects.title = 'Theseus';

```

#### 4.4 In-Context Learning (ICL)

While zero-shot learning allows models to attempt tasks without prior examples, ICL significantly enhances performance by providing demonstrations within the prompt [10]. In the context of NL-to-SQL, the objective is to maximize the probability of the model  $M$  generating the correct SQL  $s^*$  for a target question  $q$  and database schema  $\mathcal{S}$ , given a set of selected examples  $\mathcal{Q}'$  and the system instruction  $\mathcal{I}$ . This is formulated as:

$$\max_{\mathcal{Q}', \sigma} P_M(s^* \mid \sigma(q, \mathcal{S}, \mathcal{I}, \mathcal{Q}')), \quad \text{s.t. } |\mathcal{Q}'| = k \text{ and } \mathcal{Q}' \subset \mathcal{Q} \quad (4)$$

where  $\sigma$  represents the prompting function. The effectiveness of ICL relies heavily on two factors: example selection and example organization.

**Example Selection.** We investigate various selection strategies established in prior benchmarks such as DAIL-SQL [19] to optimize  $\mathcal{Q}'$ . First, *random selection* serves as a baseline, which involves sampling  $k$  examples from the candidates regardless of relevance, as illustrated in Listing 1.3.

**Listing 1.3.** Example of Random Selection Strategy

```

/* Given the following database schema: */
${DATABASE_SCHEMA}

/* Answer the following: List all available activity types */
SELECT description FROM activity_types;

${TARGET_QUESTION}

```

Second, *question similarity selection* identifies examples based on the Euclidean or cosine distance between the embeddings of the target question and candidate questions, utilizing  $k$ -nearest neighbors. Listing 1.4 demonstrates an example selected based on semantic proximity to the input.

**Listing 1.4.** Example of Question Similarity Selection

```

/* Given the following database schema: */
${DATABASE_SCHEMA}

```

```

/* Answer the following: Show the total ec max contribution of the project
Artemis */
SELECT projects.ec_max_contribution FROM projects WHERE projects.title = '
Artemis';

${TARGET_QUESTION}

```

Third, to mitigate domain bias in cross-domain settings, *masked question similarity selection* involves masking specific table and column names before embedding, thereby focusing purely on query structure (see Listing 1.5).

**Listing 1.5.** Example of Masked Question Similarity Selection

```

/* Given the following database schema: */
${DATABASE_SCHEMA}

/* Answer the following: What is the description of the panel SH2? */
/* (Selected because masked structure matches target) */
SELECT description FROM erc_panels WHERE code = 'SH2';

${TARGET_QUESTION}

```

Finally, *query similarity selection* aims to retrieve examples similar to the *target SQL* rather than the natural language question. Since the target SQL is unknown, it uses a preliminary model to generate an approximate query  $s'$ , encodes it into a syntax vector, and selects examples that maximize similarity to  $s'$  while maintaining diversity [19]. Listing 1.6 shows an example selected via this syntax-aware method.

**Listing 1.6.** Example of Query Similarity Selection

```

/* Given the following database schema: */
${DATABASE_SCHEMA}

/* Answer the following: Which funding scheme has the code 'FP7'? */
/* (Selected based on similarity to predicted SQL structure) */
SELECT title FROM funding_schemes WHERE code = 'FP7';

${TARGET_QUESTION}

```

**Example Organization.** Once examples are selected, their arrangement in the prompt impacts the model’s reasoning. We categorize organization strategies into full-information organization and SQL-only organization. The *full-information organization* preserves the complete mapping context, presenting both the natural language question and the corresponding SQL query for each example, as shown in Listing 1.7. This method ensures the model understands the semantic alignment between text and code.

**Listing 1.7.** Full-Information Organization

```

/* Given the following database schema: */
${DATABASE_SCHEMA}

/* Answer the following: Show the title of projects with a total cost of
239221.6 */
SELECT projects.title FROM projects WHERE projects.total_cost = 239221.6;

/* Answer the following: Show the names of people having projects with a
total cost != 4538143 */

```



```
SELECT people.full_name FROM people JOIN projects ON people.unics_id =
  projects.principal_investigator WHERE projects.total_cost != 4538143.0;

${TARGET_QUESTION}
```

Conversely, the *SQL-only organization* includes only the SQL queries prefaced by a brief instruction (Listing 1.8). While SQL-only organization removes the direct question-to-SQL mapping, it significantly reduces token usage, allowing the inclusion of a larger number of examples ( $k$ ) within the context window.

**Listing 1.8.** SQL-Only Organization

```
/* Some SQL examples are provided based on similar problems: */
SELECT projects.title FROM projects WHERE projects.total_cost = 239221.6;

SELECT people.full_name FROM people JOIN projects ON people.unics_id =
  projects.principal_investigator WHERE projects.total_cost != 4538143.0;

${TARGET_QUESTION}
```

## 5 Experiments

### 5.1 Dataset

We evaluate our framework using the CORDIS database (version 2022-08) from ScienceBenchmark [2]. This real-world dataset comprises 19 tables and 82 columns, with an average of 35,355 rows per table (approx. 1 GB), reflecting a complex enterprise schema. Unlike simple benchmarks, CORDIS poses distinct semantic challenges due to specialized domain terminology (e.g., “NUTS” codes) and text-heavy descriptions. We use the development set for the purpose of evaluation as the test set is not released. Furthermore, we utilize the synthetic dataset provided by the benchmark as a retrieval pool for few-shot demonstrations.

### 5.2 Baselines

To assess the effectiveness of our framework, we compare it against established baselines. We refer to the performance metrics reported in the ScienceBenchmark study [2] and related benchmarks to ensure a fair comparison under standard settings. The baselines are categorized into two groups:

#### 1. Fine-tuned NL-to-SQL approaches:

- *ValueNet* [2]: A neural architecture that explicitly incorporates value information from natural language to enhance SQL generation accuracy.
- *T5-Large+Picard* [13]: A unified text-to-text transformer fine-tuned for SQL generation. We report the performance of T5-Large serving as a strong sequence-to-sequence baseline.
- *SmBoP+GraPPa* [14]: A semi-autoregressive bottom-up semantic parser (SmBoP) combined with a pre-trained language representation model (GraPPa), representing state-of-the-art performance in schema linking and structural understanding.

2. LLM-based approaches (zero-shot & few-shot):

- *GPT-3.5 (175B)*: A powerful commercial LLM evaluated using prompting strategies, serving as a benchmark for general-purpose reasoning and code generation capabilities without task-specific fine-tuning.
- *LLaMA-2 (70B)* [20]: A state-of-the-art open-source LLM developed by Meta. It serves as a strong representative for open-weights models, offering competitive reasoning capabilities compared to proprietary systems.

### 5.3 Evaluation Metrics

Following the standard protocols established by the Spider benchmark [1], we evaluate our system using two primary accuracy metrics and a set of efficiency metrics.

**Exact Set Match Accuracy (EM).** EM measures whether the predicted SQL query  $\hat{Y}$  matches the ground truth query  $Y^*$  exactly token-for-token, ignoring whitespace. Formally:

$$Acc_{EM} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{Y}_i = Y_i^*) \quad (5)$$

where  $N$  is the total number of test samples and  $\mathbb{I}$  is the indicator function. While useful for checking syntactic adherence, EM is often too strict as it penalizes valid SQL variations.

**Execution Accuracy (EX).** Given the semantic complexity of NL-to-SQL, EX is our primary metric. It compares the execution results of the predicted query against the ground truth on the actual database instance  $D$ .

$$Acc_{EX} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{Exec}(\hat{Y}_i, D) = \text{Exec}(Y_i^*, D)) \quad (6)$$

EX is considered a more robust measure of the model’s understanding, as it accounts for different SQL formulations that yield the same data result.

**Resource Efficiency.** To assess the feasibility of deploying SLMs on resource-constrained devices, we measure:

- *Peak GPU Memory (VRAM)*: The maximum video memory allocated during inference.
- *Inference Latency*: The average time taken to generate a single SQL query (measured in seconds).

## 5.4 Implementation Details

**Hardware and Software Configuration.** All experiments were conducted on a single consumer-grade GPU to simulate a resource-constrained environment. The framework is implemented using PyTorch and the Hugging Face Transformers library. To implement the vector store and enable semantic retrieval, we employ FAISS [27] in conjunction with the *BAAI/bge-small-en* embedding model [25].

**Model and Quantization.** We use the Phi-3 model [8], a high-performance SLM quantized to 4-bit precision using QLoRA [21] techniques. This reduces memory usage significantly while maintaining reasoning capabilities.

**In-Context Learning Setup.** To analyze the impact of prompt engineering, we evaluate four distinct example selection strategies across different numbers of few-shot examples ( $k \in \{0, 1, 3, 5\}$ ):

- *Random*: Randomly selecting  $k$  examples.
- *Question Similarity* ( $QTS_S$ ): Retrieving examples based on question embedding similarity.
- *Masked Question Similarity* ( $MQS_S$ ): Retrieving examples based on masked question embeddings (hiding domain entities).
- *Query Similarity* ( $QRS_S$ ): Retrieving examples based on similarity to a predicted pseudo-SQL query.

The temperature is set to 0 for deterministic code generation, and the maximum output token length is limited to 256 tokens.

## 5.5 Results

We evaluated the performance of our SLM-based framework against established baselines on the CORDIS dataset. Table 1 summarizes the comparison in terms of execution accuracy.

**Table 1.** Performance comparison on the CORDIS dataset.

| Model                                      | Type       | EX (%)      |
|--------------------------------------------|------------|-------------|
| SmBoP + GraPPa                             | Fine-tuned | 24.0        |
| T5-Large w/o Picard                        | Fine-tuned | 29.0        |
| ValueNet                                   | Fine-tuned | 35.0        |
| LLaMA2 (70B) (5-shot)                      | LLM        | 18.0        |
| GPT-3.5 (175B) (5-shot)                    | LLM        | 57.0        |
| Our SLM (Phi-3 4-bit) + Random             | SLM        | 36.0        |
| Our SLM (Phi-3 4-bit) + Semantic Retrieval | SLM        | <b>41.0</b> |

As shown in Table 1, our approach utilizing semantic retrieval achieves an execution accuracy of 41.0%, demonstrating the effectiveness of adapting SLMs for complex domain-specific tasks.

- *Comparison with Fine-Tuned Baselines:* Our framework outperforms all specialized fine-tuned models, surpassing SmBoP + GraPPa (24.0%), T5-Large (29.0%), and ValueNet (35.0%). This indicates that a general-purpose SLM equipped with retrieval augmentation can handle the complex CORDIS schema more effectively than traditional parsers, which often struggle with generalization in low-resource settings.
- *Comparison with LLMs:* Remarkably, our 3.8B parameter model (4-bit quantized) significantly outperforms LLaMA-2 (70B), which only achieves 18.0% accuracy. While there remains a performance gap compared to the proprietary GPT-3.5 (57.0%), our model offers a compelling trade-off by running locally with minimal resources, proving that parameter count is not the sole determinant of success in domain-specific NL-to-SQL tasks.

**Ablation Study.** To understand the factors driving this performance, we analyzed different ICL strategies and the sensitivity to the number of examples ( $k$ ). The results are detailed in Table 2.

**Table 2.** Execution accuracy (%) of the SLM across different example selection strategies and number of shots ( $k$ ).

| Strategy                         | $k=0$ | $k=1$       | $k=3$       | $k=5$ |
|----------------------------------|-------|-------------|-------------|-------|
| Random                           | 36.0  | 37.0        | 40.0        | 35.0  |
| Question Similarity ( $QTS_S$ )  | 36.0  | <b>41.0</b> | <b>41.0</b> | 34.0  |
| Masked Question Sim. ( $MQS_S$ ) | 35.0  | 37.0        | 37.0        | 37.0  |
| Query Similarity ( $QRS_S$ )     | 35.0  | 37.0        | 37.0        | 34.0  |

- *Impact of Selection Strategies:* The results highlight that the standard  $QTS_S$  strategy is the most effective, peaking at 41.0% for  $k=1$  and  $k=3$ . This suggests that retrieving examples based on direct natural language similarity best helps the SLM map user intent to SQL logic. Interestingly, complex methods like  $MQS_S$  and  $QRS_S$  plateau at 37.0%, underperforming even the Random baseline at  $k=3$  (40.0%). This implies that masking entities or relying on pseudo-SQL structures may remove critical context or introduce noise in this specific dataset.
- *Sensitivity to Number of Examples ( $k$ ):* We observe a distinct “inverted-U” performance trend. Accuracy improves significantly from zero-shot ( $k=0$ ) to few-shot ( $k=1, 3$ ), confirming the value of ICL. However, performance degrades at  $k=5$  across most strategies (e.g.,  $QTS_S$  drops to 34.0%). This decline is likely due to the “lost-in-the-middle” phenomenon or context window limitations of the SLM, where excessive tokens from 5 examples dilute the model’s attention on the target query. Therefore, the range of  $k=1$  to  $k=3$  represents the optimal configuration for this architecture.

**Efficiency Evaluation.** A key advantage of our framework is resource efficiency. Experiments were conducted on a single consumer-grade GPU. The 4-bit quantized SLM requires approximately 3.8 GB of VRAM. Even with the retrieval context, peak memory usage remains under 6 GB, enabling deployment on edge devices. In contrast, GPT-3.5 relies on external APIs with associated costs and latency, while LLaMA-2 (70B) necessitates high-end hardware (e.g., A100 clusters) for inference, yet yields lower accuracy on this task.

**Error Analysis.** To identify remaining challenges, we examined failure cases of the best-performing model ( $QTS_S, k = 3$ ):

- *Schema Linking Ambiguity:* The majority of errors arise from selecting incorrect columns in semantically similar fields (e.g., distinguishing *ec\_contribution* vs. *total\_cost* in the *projects* table).
- *Complex Nesting:* While  $QTS_S$  retrieves semantically similar questions, the corresponding SQL logic sometimes requires complex nested structures. In these cases, the SLM occasionally fails to generate syntactically correct code, despite correctly detecting the user’s intent.
- *Context Sensitivity:* The performance drop at  $k = 5$  confirms the model’s sensitivity to prompt length, suggesting that future work should focus on prompt compression rather than simply increasing the number of shots.

## 5.6 Discussion

The results of this study suggest a paradigm shift in how we approach domain-specific NL-to-SQL tasks. Historically, high performance was synonymous with either computationally expensive fine-tuning of semantic parsers (e.g., ValueNet [2]) or the use of massive proprietary models. Our findings challenge this assumption in two key ways:

- *Small vs. Large Open Models:* Most notably, our 4-bit quantized SLM ( $\sim 3.8$ B parameters) significantly outperforms LLaMA-2 (70B), achieving 41.0% accuracy versus 18.0%. This indicates that parameter count is not the sole determinant of reasoning capability. A small, focused model with effective retrieval augmentation can outperform a massive general-purpose model that lacks specific adaptation to the schema.
- *Generalization vs. Specialization:* By surpassing fine-tuned baselines (24.0% - 35.0%) without any weight updates, we demonstrate that general-purpose reasoning capabilities in SLMs, when guided by precise in-context examples, offer better generalization on complex schemas than traditional supervised training, which often suffers from overfitting to training set distributions.

While our framework offers a compelling alternative, the comparison with GPT-3.5 (57.0%) highlights distinct trade-offs.

- *The Efficacy of Simplicity:* Our ablation study reveals that simpler retrieval strategies ( $QTS_S$ ) outperform complex ones ( $MQS_S, QRS_S$ ). This suggests

that SLMs benefit more from clear, direct natural language analogies than from noisy, pseudo-generated SQL context. Over-engineering the retrieval process can be counterproductive for smaller models with limited attention heads.

- *Context Window Limitations*: The performance degradation at  $k = 5$  (dropping from 41.0% to 34.0%) confirms that SLMs are highly sensitive to “information overload”. Unlike commercial LLMs that can handle long contexts, SLMs require a delicate balance: the context must be rich enough to guide, but concise enough to avoid the “lost-in-the-middle” phenomenon.
- *The Performance Gap*: While we achieve state-of-the-art results among open-weights and fine-tuned models, the gap with GPT-3.5 indicates that for scenarios requiring the highest possible accuracy regardless of cost or privacy, proprietary giants remain superior. Our solution is better positioned for scenarios prioritizing privacy, latency, and local execution.

## 6 Conclusion

In this work, we introduced Light-SQL, demonstrating that the democratization of natural language interfaces for databases does not rely solely on massive proprietary models, but rather on the intelligent orchestration of SLMs and semantic retrieval strategies. Our experiments on the CORDIS dataset show that a localized, 3.8B parameter model can achieve 41.0% execution accuracy, effectively outperforming the 70B-parameter LLaMA-2 and specialized fine-tuned parsers. The study establishes that the key to unlocking SLM potential lies in the quality of context rather than the sheer quantity of parameters. We identified that a direct question-similarity retrieval strategy ( $QTS_S$ ) significantly enhances reasoning capabilities, whereas more complex masking techniques yield diminishing returns. Furthermore, our framework ensures data sovereignty and operational efficiency, operating seamlessly on a single consumer GPU (<6 GB VRAM), thereby addressing the critical barriers of latency and privacy in sensitive domains such as finance and healthcare.

In future work, we plan to address the context window limitations observed in our ablation studies. We will incorporate chain-of-thought prompting and self-correction mechanisms to mitigate the “lost-in-the-middle” phenomenon, aiming to approach the reasoning depth of commercial LLMs while retaining the efficiency benefits of edge deployment.

## References

1. Yu, T., et al.: Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (2018)
2. Zhang, Y., et al.: Sciencebenchmark: a complex real-world benchmark for evaluating natural language to SQL systems. In: Proceedings of the VLDB Endowment (2023)

3. Li, J., et al.: Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs. In: *Advances in Neural Information Processing Systems*, vol. 36, 42330–42357 (2023)
4. Zhao, W.X., et al.: A survey of large language models. *arXiv preprint [arXiv:2303.18223](https://arxiv.org/abs/2303.18223)* 1.2 (2023)
5. OpenAI, R.: Gpt-4 technical report. *arxiv* 2303.08774. View in Article **2**(5), 1 (2023)
6. Jiang, A.Q., et al.: Mistral 7B. *arXiv preprint [arXiv:2310.06825](https://arxiv.org/abs/2310.06825)* (2023)
7. Zhang, P., Zeng, G., Wang, T., Lu, W.: TinyLlama: an open-source small language model. *arXiv preprint [arXiv:2401.02385](https://arxiv.org/abs/2401.02385)* (2024)
8. Abdin, M., et al.: Phi-3 technical report: a highly capable language model locally on your phone. *arXiv preprint [arXiv:2404.14219](https://arxiv.org/abs/2404.14219)* (2024)
9. Liu, P., et al.: Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. *ACM Comput. Surv.* **55**(9), 1–35 (2023)
10. Brown, T., et al.: Language models are few-shot learners. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901 (2020)
11. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837 (2022)
12. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474 (2020)
13. Raffel, C., et al.: Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* **21**(140), 1–67 (2020)
14. Rubin, O., Berant, J.: SmBoP: semi-autoregressive bottom-up semantic parsing. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (2021)
15. Fan, Y., et al.: MetaSQL: a generate-then-rank framework for natural language to SQL translation. In: *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE (2024)
16. Wang, B., et al.: “Rat-SQL: relation-aware schema encoding and linking for text-to-SQL parsers. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (2020)
17. Scholak, T., Schucher, N., Bahdanau, D.: PICARD: parsing incrementally for constrained auto-regressive decoding from language models. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (2021)
18. Pourreza, M., Rafiei, D.: Din-SQL: Decomposed in-context learning of text-to-SQL with self-correction. *Adv. Neural. Inf. Process. Syst.* **36**, 36339–36348 (2023)
19. Gao, D., Wang, H., Li, Y., et al.: Text-to-SQL empowered by large language models: a benchmark evaluation. *Proc. VLDB Endowment* (2024)
20. Touvron, H., et al.: Llama 2: open foundation and fine-tuned chat models. *arXiv preprint [arXiv:2307.09288](https://arxiv.org/abs/2307.09288)* (2023)
21. Dettmers, T., et al.: Qlora: efficient finetuning of quantized LLMs. *Adv. Neural. Inf. Process. Syst.* **36**, 10088–10115 (2023)
22. Wang, X., et al.: Self-consistency improves chain of thought reasoning in language models. *arXiv preprint [arXiv:2203.11171](https://arxiv.org/abs/2203.11171)* (2022)
23. Karpukhin, V., et al.: Dense passage retrieval for open-domain question answering. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (2020)

24. Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (2019)
25. Xiao, S., Liu, Z., Zhang, P., Muennighoff, N.: C-Pack: packaged resources to advance general chinese embedding. arXiv preprint [arXiv:2309.07597](https://arxiv.org/abs/2309.07597) (2023)
26. Lei, W., et al.: Re-examining the role of schema linking in Text-to-SQL. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP) (2020)
27. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. *IEEE Trans. Big Data* **7**(3), 535–547 (2019)